# TEAM CROISSANT

## MoMo SMS Data Pipeline

Database Design Document

May 2026

---


**Team Members**

Nshuti Lancelot — Schema Lead

Imanzi Beni — Repository Lead

Ishimwe Axcel — ERD Diagram

Teta Dianah — System Logs & Validation

Rugwiro Derrick — JSON, Integration Tests & PDF

# Table of Contents

# 1. ERD Diagram

The Entity Relationship Diagram below was designed by Ishimwe Axcel. It shows all five tables in the momo_db database and how they relate to each other through primary and foreign keys.

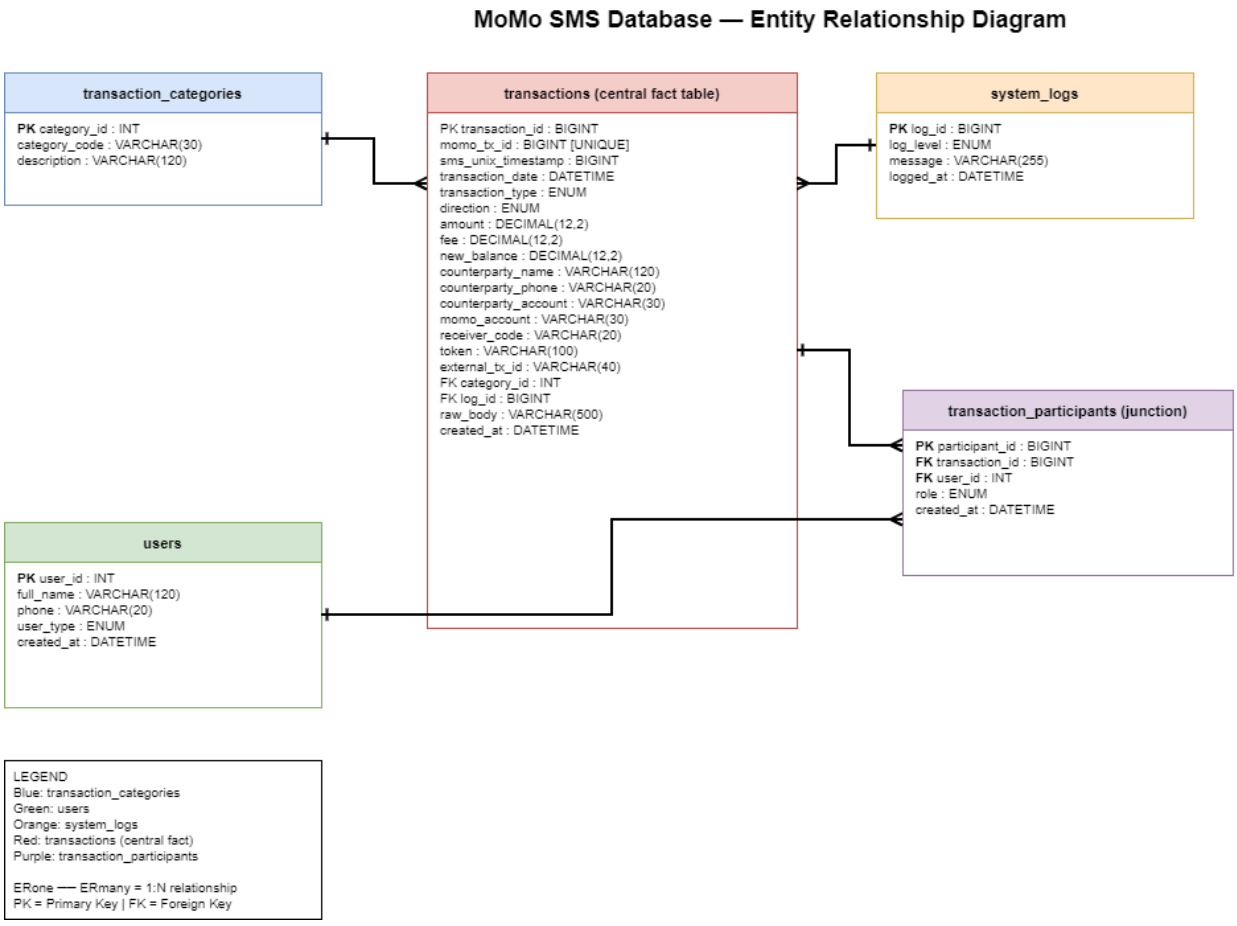



*Figure 1: Entity Relationship Diagram — momo_db (designed by Ishimwe Axcel)*

## Key Relationships

- Transactions is the central fact table. Every transaction record must belong to a category, making category_id a required foreign key that references Transaction_Categories. This ensures no transaction is stored without a known type.
- Transactions optionally references System_Logs via log_id. When the ETL pipeline parses an SMS it can attach a log entry to the transaction, recording whether parsing succeeded, produced a warning, or failed with an error.
- Transaction_Participants is a junction table that sits between Transactions and Users. It resolves the many-to-many relationship — one transaction can involve multiple users (a sender and a receiver), and one user can appear in many transactions. Each row in Transaction_Participants records which user played which role in which transaction.
- Users are not stored directly inside Transactions. Instead the Users table holds a clean list of every unique person who has ever appeared as a sender or receiver, and

Transaction_Participants links them to the relevant transactions. This avoids duplicating user data across every transaction record.

- ON DELETE CASCADE is applied to Transaction_Participants. When a transaction is deleted from the Transactions table, all participant records linked to that transaction are automatically deleted as well. This keeps the database consistent and prevents orphaned records from accumulating.
- Transaction_Categories uses a UNIQUE constraint on category_code. This prevents duplicate category types from being inserted and ensures every transaction is mapped to exactly one distinct category such as received, transfer, airtime, bank_deposit, or direct_payment.
- System_Logs uses a CHECK constraint (chk_log_message_not_empty) that rejects any log entry with an empty message. This ensures every log record contains meaningful information and cannot be inserted as a blank placeholder.

# 2. Data Dictionary

The database momo_db contains 5 tables designed to store and relate MoMo SMS transaction data. Each table was built by a different team member.

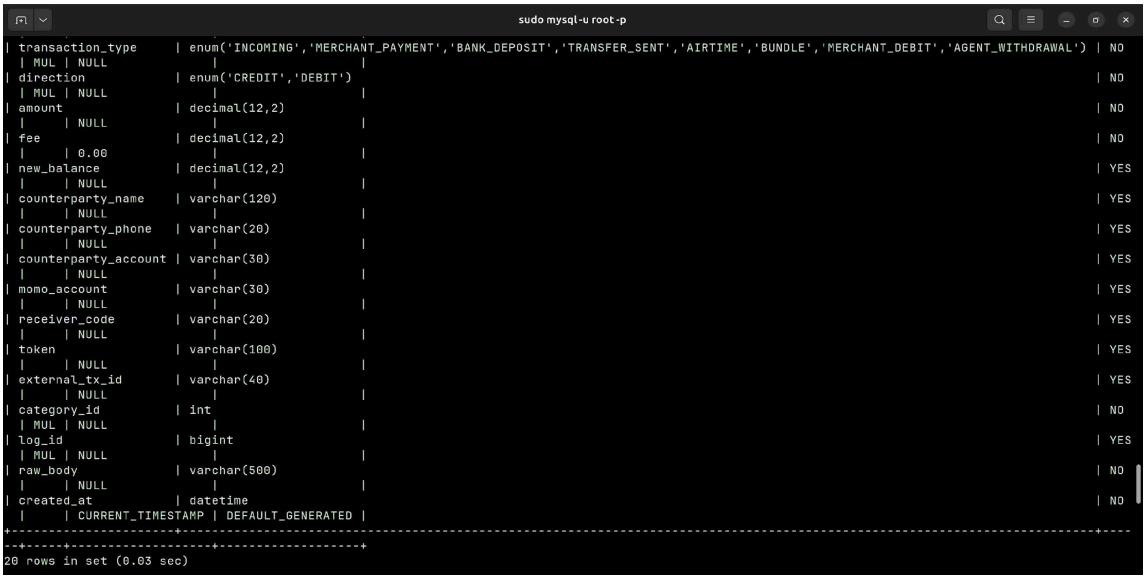
## 2.1 Transactions

The central fact table. Stores every parsed MoMo SMS transaction including amount, direction, category, and the raw SMS body. Designed by Nshuti Lancelot.

| Column | Type | Nullable | Description |
|---|---|---|---|
| transaction_id | `BIGINT AUTO_INCREMENT` | NO | Primary key |
| momo_tx_id | `BIGINT UNIQUE` | YES | MTN Financial Transaction ID from SMS |
| sms_unix_timestamp | `BIGINT` | NO | Unix timestamp from SMS metadata |
| transaction_date | `DATETIME` | NO | Date and time of the transaction |
| transaction_type | `ENUM(...)` | NO | INCOMING, TRANSFER_SENT, AIRTIME etc. |
| direction | `ENUM('CREDIT','DEBIT')` | NO | CREDIT = money in, DEBIT = money out |
| amount | `DECIMAL(12,2)` | NO | Transaction amount in RWF |
| fee | `DECIMAL(12,2)` | NO | Fee charged (default 0.00) |
| new_balance | `DECIMAL(12,2)` | YES | Account balance after the transaction |
| counterparty_name | `VARCHAR(120)` | YES | Name of the sender or recipient |
| counterparty_phone | `VARCHAR(20)` | YES | Phone number of the counterparty |
| category_id | `INT` | NO | Foreign key → Transaction_Categories |
| log_id | `BIGINT` | YES | Foreign key → System_Logs |
| raw_body | `VARCHAR(500)` | NO | Original SMS message text |
| created_at | `DATETIME` | NO | Timestamp when record was inserted |

Transactions table schema screenshots from Nshuti Lancelot:


*Transactions table schema — part 1 (Lancelot)*




*Transactions table schema — part 2 (Lancelot)*

### 2.2 Transaction_Categories

Lookup table classifying each transaction into a type such as received, transfer, airtime, bank_deposit. Designed by Ishimwe Axcel.

| Column | Type | Nullable | Description |
|---|---|---|---|
| category_id | `INT AUTO_INCREMENT` | NO | Primary key |
| category_code | `VARCHAR(30) UNIQUE` | NO | Short code e.g. received, transfer, airtime |
| description | `VARCHAR(120)` | YES | Human-readable description of the category |

### 2.3 Users

Stores every unique sender and receiver parsed from the SMS messages. Designed by Imanzi Beni.

| Column | Type | Nullable | Description |
|---|---|---|---|
| user_id | `INT AUTO_INCREMENT` | NO | Primary key |
| full_name | `VARCHAR(120)` | NO | Full name extracted from SMS body |
| phone | `VARCHAR(20)` | YES | Phone number (may be partially masked) |
| user_type | `ENUM('CUSTOMER','AGENT','MERCHANT')` | NO | Role of the user in the system |
| created_at | `DATETIME` | NO | Timestamp when record was inserted |

### 2.4 System_Logs

Tracks parsing events, warnings and errors generated during the ETL pipeline. Each log entry can be linked to a transaction. Designed by Teta Dianah.

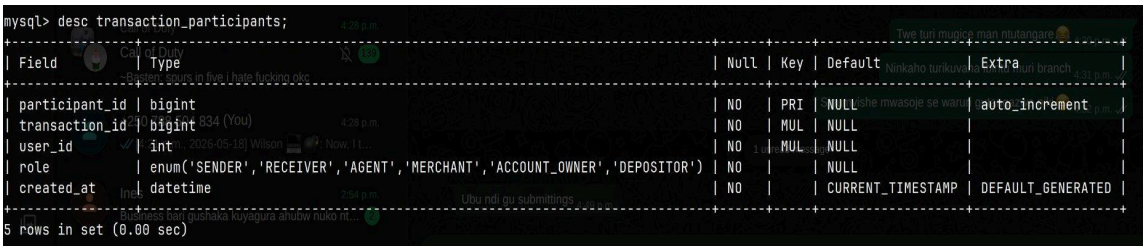
| Column | Type | Nullable | Description |
|---|---|---|---|
| log_id | `BIGINT AUTO_INCREMENT` | NO | Primary key |
| log_level | `ENUM('INFO','WARNING','ERROR')` | NO | Severity level of the log entry |
| message | `VARCHAR(255)` | NO | Log message — cannot be empty (CHECK constraint) |
| logged_at | `DATETIME` | NO | Timestamp when the log was created |

### 2.5 Transaction_Participants

Junction table linking Users to Transactions with a role (SENDER or RECEIVER). When a transaction is deleted, its participants are automatically deleted via CASCADE. Designed by Nshuti Lancelot.

| Column | Type | Nullable | Description |
|---|---|---|---|
| participant_id | `BIGINT AUTO_INCREMENT` | NO | Primary key |
| transaction_id | `BIGINT` | NO | FK → Transactions (ON DELETE CASCADE) |
| user_id | `INT` | NO | Foreign key → Users |
| role | `ENUM('SENDER','RECEIVER','AGENT','MERCHANT','ACCOUNT_OWNER','DEPOSITOR')` | NO | Role of the user in this transaction |
| created_at | `DATETIME` | NO | Timestamp when record was inserted |

Transaction_Participants schema screenshot from Nshuti Lancelot:



```
mysql> desc transaction_participants;
+----------------+-----------------------------------------------------------------------+------+-----+-------------------+-------------------+
| Field          | Type                                                                  | Null | Key | Default           | Extra             |
+----------------+-----------------------------------------------------------------------+------+-----+-------------------+-------------------+
| participant_id | bigint                                                                | NO   | PRI | NULL              | auto_increment    |
| transaction_id | bigint                                                                | NO   | MUL | NULL              |                   |
| user_id        | int                                                                   | NO   | MUL | NULL              |                   |
| role           | enum('SENDER','RECEIVER','AGENT','MERCHANT','ACCOUNT_OWNER','DEPOSITOR') | NO   |     | NULL              |                   |
| created_at     | datetime                                                              | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+----------------+-----------------------------------------------------------------------+------+-----+-------------------+-------------------+
5 rows in set (0.00 sec)
```

*Transaction_Participants table schema (Lancelot)*

# 3. Demo Query Screenshots

Three cross-table analytics queries demonstrating the database capabilities. These queries were designed by Rugwiro Derrick and use data from multiple tables.

## Query 1 — Total Transactions per Category

This query joins Transactions with Transaction_Categories and groups results by category to show total transaction count and total amount in RWF for each category type.

```
SELECT c.category_code, c.description,
       COUNT(t.transaction_id) AS transaction_count,
       SUM(t.amount) AS total_amount_rwf
FROM transactions t
JOIN transaction_categories c ON t.category_id = c.category_id
GROUP BY c.category_code ORDER BY total_amount_rwf DESC;
```

```
+---------------+-----------------------------------------+-------------------+------------------+
| category_code | description                             | transaction_count | total_amount_rwf |
+---------------+-----------------------------------------+-------------------+------------------+
| received      | money received from another momo user   |                 1 |          2000.00 |
| payment       | outgoing payment to another person      |                 1 |          1000.00 |
| transfer      | money transfer to a phone number        |                 1 |         10000.00 |
| bank_deposit  | cash deposited into momo from bank      |                 1 |         40000.00 |
| airtime       | Airtime purchase using MoMo balance     |                 1 |          2000.00 |
| direct_payment| Payment processed by a merchant or business |             1 |           600.00 |
| withdrawal    | Cash withdrawn from MoMo via an agent   |                 0 |             NULL |
| bank_transfer | Transfer from MoMo to a bank account    |                 0 |             NULL |
+---------------+-----------------------------------------+-------------------+------------------+
8 rows in set (0.01 sec)
```

*Query 1 result — 8 categories with transaction counts and totals*

## Query 2 — All System Log Entries

This query reads all records from System_Logs showing every parsing event recorded during the ETL pipeline run across 1,688 SMS messages.

```
SELECT * FROM system_logs;
```

```
mysql> SELECT * FROM system_logs;
+--------+-----------+-----------------------------------------------------------+---------------------+
| log_id | log_level | message                                                   | logged_at           |
+--------+-----------+-----------------------------------------------------------+---------------------+
|      1 | INFO      | Successfully parsed SMS at index 1 - received transaction | 2026-05-18 09:18:51 |
|      2 | INFO      | Successfully parsed SMS at index 2 - payment transaction  | 2026-05-18 09:18:51 |
|      3 | WARNING   | SMS at index 45 matched no known pattern - skipped        | 2026-05-18 09:18:51 |
|      4 | ERROR     | SMS at index 67 - malformed date field, could not parse   | 2026-05-18 09:18:51 |
|      5 | INFO      | Pipeline completed - 1688 records processed, 3 skipped    | 2026-05-18 09:18:51 |
+--------+-----------+-----------------------------------------------------------+---------------------+
5 rows in set (0.01 sec)
```

*Query 2 result — all system log entries including INFO, WARNING and ERROR levels*

## Query 3 — Errors and Warnings Only

This query filters System_Logs to show only ERROR and WARNING entries, helping identify which SMS messages failed to parse correctly.

```
SELECT * FROM system_logs WHERE log_level IN ('ERROR', 'WARNING');
```

```
mysql> SELECT * FROM system_logs WHERE log_level IN ('ERROR', 'WARNING');
+--------+-----------+-----------------------------------------------------+---------------------+
| log_id | log_level | message                                             | logged_at           |
+--------+-----------+-----------------------------------------------------+---------------------+
|      3 | WARNING   | SMS at index 45 matched no known pattern - skipped  | 2026-05-18 09:18:51 |
|      4 | ERROR     | SMS at index 67 - malformed date field, could not parse | 2026-05-18 09:18:51 |
+--------+-----------+-----------------------------------------------------+---------------------+
2 rows in set (0.00 sec)
```

*Query 3 result — 2 rows: 1 WARNING and 1 ERROR from the pipeline run*

# 4. Validation Rule Screenshots

Validation rules and CRUD operation screenshots from Teta Dianah (System_Logs), Imanzi Beni (Users), and Ishimwe Axcel (Transaction_Categories). These confirm that data integrity constraints work correctly across the schema.

## 4.1 System_Logs — CHECK Constraint (Teta Dianah)

A CHECK constraint was added to System_Logs to prevent empty messages from being inserted. The screenshots below show the constraint being created and then correctly rejecting a bad INSERT.

```
mysql> ALTER TABLE system_logs
    ->   ADD CONSTRAINT chk_log_message_not_empty CHECK (LENGTH(message) > 0);
Query OK, 5 rows affected (0.10 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

*ALTER TABLE — adding chk_log_message_not_empty CHECK constraint, 5 rows affected*

```
mysql> INSERT INTO system_logs (log_level, message) VALUES ('INFO', '');
ERROR 3819 (HY000): Check constraint 'chk_log_message_not_empty' is violated.
mysql>
```

*Constraint working — INSERT with empty message is rejected with ERROR 3819*

## 4.2 System_Logs — CRUD Operations (Teta Dianah)

Full Create, Read, Update, Delete cycle on the System_Logs table demonstrating all operations work correctly.

```
+--------+-----------+-------------------------------------------------------+---------------------+
| log_id | log_level | message                                               | logged_at           |
+--------+-----------+-------------------------------------------------------+---------------------+
|      1 | INFO      | Successfully parsed SMS at index 1 - received transaction | 2026-05-17 23:56:36 |
|      2 | INFO      | Successfully parsed SMS at index 2 - payment transaction  | 2026-05-17 23:56:36 |
|      3 | WARNING   | SMS at index 45 matched no known pattern - skipped        | 2026-05-17 23:56:36 |
|      4 | ERROR     | SMS at index 67 - malformed date field, could not parse   | 2026-05-17 23:56:36 |
|      5 | INFO      | Pipeline completed - 1688 records processed, 3 skipped    | 2026-05-17 23:56:36 |
|      6 | WARNING   | Test log entry for CRUD verification                      | 2026-05-17 23:56:36 |
+--------+-----------+-------------------------------------------------------+---------------------+
6 rows in set (0.00 sec)
```

*CREATE — 6 log entries inserted including test entry for CRUD verification*

```
+--------+-----------+-----------------------------------------------------+---------------------+
| log_id | log_level | message                                             | logged_at           |
+--------+-----------+-----------------------------------------------------+---------------------+
|      1 | INFO      | Successfully parsed SMS at index 1 - received transaction | 2026-05-17 23:56:36 |
|      2 | INFO      | Successfully parsed SMS at index 2 - payment transaction  | 2026-05-17 23:56:36 |
|      3 | WARNING   | SMS at index 45 matched no known pattern - skipped        | 2026-05-17 23:56:36 |
|      4 | ERROR     | SMS at index 67 - malformed date field, could not parse   | 2026-05-17 23:56:36 |
|      5 | INFO      | Pipeline completed - 1688 records processed, 3 skipped    | 2026-05-17 23:56:36 |
+--------+-----------+-----------------------------------------------------+---------------------+
5 rows in set (0.00 sec)
```

*DELETE — test log entry removed, 5 rows remain*

```
+--------+-----------+-----------------------------------------------------+---------------------+
| log_id | log_level | message                                             | logged_at           |
+--------+-----------+-----------------------------------------------------+---------------------+
|      3 | WARNING   | SMS at index 45 matched no known pattern - skipped | 2026-05-17 23:56:36 |
|      6 | WARNING   | Updated test log entry                              | 2026-05-17 23:56:36 |
+--------+-----------+-----------------------------------------------------+---------------------+
2 rows in set (0.00 sec)
```

*READ — SELECT WARNING logs only returns 2 rows*

### 4.3 Users — CRUD Operations (Imanzi Beni)

Full CRUD cycle on the Users table. A test user was inserted, verified, updated, and deleted, confirming all operations work correctly.

```
mysql> INSERT INTO users (full_name, phone, user_type)
    -> VALUES ('Test User', '250799999999', 'CUSTOMER');
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM users;
+---------+---------------+--------------+-----------+---------------------+
| user_id | full_name     | phone        | user_type | created_at          |
+---------+---------------+--------------+-----------+---------------------+
|       1 | Jane Smith    | 250781111111 | CUSTOMER  | 2026-05-16 18:07:32 |
|       2 | Samuel Carter | 250791666666 | CUSTOMER  | 2026-05-16 18:07:32 |
|       3 | Linda Green   | 250788123456 | MERCHANT  | 2026-05-16 18:07:32 |
|       4 | Alex Doe      | 250722345678 | AGENT     | 2026-05-16 18:07:32 |
|       5 | Robert Brown  | 250733456789 | CUSTOMER  | 2026-05-16 18:07:32 |
|       7 | Test User     | 250799999999 | CUSTOMER  | 2026-05-18 15:41:33 |
+---------+---------------+--------------+-----------+---------------------+
6 rows in set (0.00 sec)
```

*CREATE — Test User inserted, SELECT shows 6 rows including new user*

```
mysql> UPDATE users SET phone = '250781111111' WHERE full_name = 'Jane Smith';
Query OK, 0 rows affected (0.00 sec)
Rows matched: 1  Changed: 0  Warnings: 0

mysql> SELECT * FROM users WHERE full_name = 'Jane Smith';
+---------+------------+--------------+-----------+---------------------+
| user_id | full_name  | phone        | user_type | created_at          |
+---------+------------+--------------+-----------+---------------------+
|       1 | Jane Smith | 250781111111 | CUSTOMER  | 2026-05-16 18:07:32 |
+---------+------------+--------------+-----------+---------------------+
1 row in set (0.00 sec)
```

*UPDATE — Jane Smith phone number updated and verified with SELECT*

```
mysql> DELETE FROM users WHERE full_name = 'Test User';
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM users;
+---------+--------------+-------------+-----------+---------------------+
| user_id | full_name    | phone       | user_type | created_at          |
+---------+--------------+-------------+-----------+---------------------+
|       1 | Jane Smith   | 250781111111 | CUSTOMER  | 2026-05-16 18:07:32 |
|       2 | Samuel Carter | 250791666666 | CUSTOMER  | 2026-05-16 18:07:32 |
|       3 | Linda Green  | 250788123456 | MERCHANT  | 2026-05-16 18:07:32 |
|       4 | Alex Doe     | 250722345678 | AGENT     | 2026-05-16 18:07:32 |
|       5 | Robert Brown | 250733456789 | CUSTOMER  | 2026-05-16 18:07:32 |
+---------+--------------+-------------+-----------+---------------------+
5 rows in set (0.00 sec)
```

*DELETE — Test User removed, SELECT confirms 5 rows remain*

```
mysql> SELECT * FROM users;
+---------+--------------+-------------+-----------+---------------------+
| user_id | full_name    | phone       | user_type | created_at          |
+---------+--------------+-------------+-----------+---------------------+
|       1 | Jane Smith   | 250781111111 | CUSTOMER  | 2026-05-16 18:07:32 |
|       2 | Samuel Carter | 250791666666 | CUSTOMER  | 2026-05-16 18:07:32 |
|       3 | Linda Green  | 250788123456 | MERCHANT  | 2026-05-16 18:07:32 |
|       4 | Alex Doe     | 250722345678 | AGENT     | 2026-05-16 18:07:32 |
|       5 | Robert Brown | 250733456789 | CUSTOMER  | 2026-05-16 18:07:32 |
+---------+--------------+-------------+-----------+---------------------+
5 rows in set (0.01 sec)
```

*READ — Final SELECT all users shows 5 clean records*

### 4.4 Transaction_Categories — CRUD Operations (Ishimwe Axcel)

Full CRUD cycle on the Transaction_Categories table. A test category was inserted, updated, and deleted, confirming all 8 real categories remain intact.

```
+-------------+---------------+-------------------------------------------+
| category_id | category_code | description                               |
+-------------+---------------+-------------------------------------------+
|           1 | received      | money received from another momo user     |
|           2 | payment       | outgoing payment to another person        |
|           3 | transfer      | money transfer to a phone number          |
|           4 | bank_deposit  | cash deposited into momo from bank        |
|           5 | airtime       | Airtime purchase using MoMo balance       |
|           6 | direct_payment | Payment processed by a merchant or business |
|           7 | withdrawal    | Cash withdrawn from MoMo via an agent     |
|           8 | bank_transfer | Transfer from MoMo to a bank account      |
|           9 | test_type     | Temporary test category                   |
+-------------+---------------+-------------------------------------------+
9 rows in set (0.00 sec)
```

*READ — SELECT all categories shows 9 rows including test_type*

```
+-------------+---------------+--------------------------+
| category_id | category_code | description              |
+-------------+---------------+--------------------------+
|           9 | test_type     | Updated test description |
+-------------+---------------+--------------------------+
1 row in set (0.00 sec)
```

*UPDATE — test_type description updated to 'Updated test description'*

```
+-------------+---------------+-----------------------------------------+
| category_id | category_code | description                             |
+-------------+---------------+-----------------------------------------+
|           1 | received      | money received from another momo user   |
|           2 | payment       | outgoing payment to another person      |
|           3 | transfer      | money transfer to a phone number        |
|           4 | bank_deposit  | cash deposited into momo from bank      |
|           5 | airtime       | Airtime purchase using MoMo balance     |
|           6 | direct_payment| Payment processed by a merchant or business |
|           7 | withdrawal    | Cash withdrawn from MoMo via an agent   |
|           8 | bank_transfer | Transfer from MoMo to a bank account    |
+-------------+---------------+-----------------------------------------+
8 rows in set (0.00 sec)
```

*DELETE — test_type removed, 8 real categories remain*

# 5. JSON Modeling & Mapping Queries

This section demonstrates how the relational database schema can be serialized into nested JSON for API consumption. This work was done by Rugwiro Derrick.

## 5.1 Complex Transaction JSON

Below is a complete MoMo transaction represented as a single nested JSON object. In a real API response, data from 4 separate SQL tables (Transactions, Transaction_Categories, Users, System_Logs) is combined into one response so the client gets everything it needs without making multiple requests.


```
{
  "transaction_id": 1,
  "momo_tx_id": 76662021700,
  "amount": 2000.00,
  "fee": 0.00,
  "new_balance": 2000.00,
  "direction": "CREDIT",
  "transaction_date": "2024-05-10T16:30:51Z",
  "raw_body": "You have received 2000 RWF from Jane Smith...",
  "category": {
    "category_id": 1,
    "category_code": "INCOMING_MONEY"
  },
  "sender": {
    "user_id": 42,
    "full_name": "Jane Smith",
    "phone": "250788000013",
    "user_type": "CUSTOMER"
  },
  "receiver": {
    "user_id": 1,
    "full_name": "Account Owner",
    "phone": "250788000001",
    "user_type": "CUSTOMER"
  },
  "log": {
    "log_id": 1,
    "log_level": "INFO",
    "message": "Transaction parsed successfully"
  }
}
```


## 5.2 SQL-to-JSON Field Mapping

The table below explains how each SQL table and its columns map to the JSON fields in the complex transaction object above.

| SQL Table | Columns | JSON Field |
|---|---|---|
| Transactions | transaction_id, amount, fee, direction... | Root level fields of the JSON object |

| | | |
|---|---|---|
| Transaction_Categories | category_id, category_code | Nested under category: { } |
| Users | user_id, full_name, phone, user_type | Nested under sender: { } or receiver: { } |
| System_Logs | log_id, log_level, message | Nested under log: { } |

### 5.3 Mapping Query

The SQL query below uses MySQL's built-in JSON_OBJECT() function to JOIN all four tables and produce the nested JSON output shown in section 5.1. This proves the relational schema can be directly serialized as JSON for an API.

```
SELECT JSON_OBJECT(
  'transaction_id', t.transaction_id,
  'amount',         t.amount,
  'direction',      t.direction,
  'category', JSON_OBJECT(
    'category_id',   c.category_id,
    'category_code', c.category_code
  ),
  'log', JSON_OBJECT(
    'log_id',    l.log_id,
    'log_level', l.log_level,
    'message',   l.message
  )
) AS transaction_json
FROM transactions t
JOIN transaction_categories c ON t.category_id = c.category_id
LEFT JOIN system_logs l ON t.log_id = l.log_id
LIMIT 1;
```

## 6. Integration Test Screenshots

A sequence of 8 cross-table CRUD operations run by Rugwiro Derrick. These tests verify that all 5 tables work together correctly — inserting data that references multiple tables, updating it, and confirming that CASCADE DELETE removes related records automatically.

### Test 1 — INSERT User

A test customer is inserted into the Users table to serve as a participant in the test transaction.

```
Database changed
mysql> INSERT INTO Users (full_name, phone, user_type)
    -> VALUES ('Test User', '250788000001', 'CUSTOMER');
Query OK, 1 row affected (0.014 sec)
```

*INSERT INTO Users — Query OK, 1 row affected (0.014 sec)*

### Test 2 — INSERT System Log

A test log entry is inserted into System_Logs. This log will be linked to the test transaction in the next step.

```
mysql> INSERT INTO System_Logs (log_level, message)
    -> VALUES ('INFO', 'Test transaction parsed successfully');
Query OK, 1 row affected (0.005 sec)
```

*INSERT INTO System_Logs — Query OK, 1 row affected (0.005 sec)*

### Test 3 — INSERT Transaction

A test transaction is inserted referencing category_id = 1 and the log created in Test 2 via LAST_INSERT_ID(). The sms_unix_timestamp is provided using UNIX_TIMESTAMP().

```
mysql> INSERT INTO Transactions (
    ->   momo_tx_id, category_id, amount, fee,
    ->   new_balance, direction, counterparty_name,
    ->   transaction_date, raw_body, log_id,
    ->   sms_unix_timestamp
    -> )
    -> VALUES (
    ->   99999999999, 1, 5000.00, 100.00,
    ->   28300.00, 'DEBIT', 'Test User',
    ->   '2024-05-11 20:34:47',
    ->   '5000 RWF transferred to Test User',
    ->   LAST_INSERT_ID(),
    ->   UNIX_TIMESTAMP('2024-05-11 20:34:47')
    -> );
Query OK, 1 row affected (0.035 sec)
```

*INSERT INTO Transactions — Query OK, 1 row affected, all FK references resolved*

## Test 4 — SELECT with JOINs

The test transaction is retrieved using JOINs across Transactions, Transaction_Categories and System_Logs. This proves the foreign key relationships work correctly.

```
mysql> SELECT
    ->   t.transaction_id,
    ->   t.amount,
    ->   t.direction,
    ->   c.category_code,
    ->   l.log_level
    -> FROM Transactions t
    -> JOIN Transaction_Categories c ON t.category_id = c.category_id
    -> LEFT JOIN System_Logs l ON t.log_id = l.log_id
    -> WHERE t.momo_tx_id = 99999999999;
+----------------+---------+-----------+---------------+-----------+
| transaction_id | amount  | direction | category_code | log_level |
+----------------+---------+-----------+---------------+-----------+
|              8 | 5000.00 | DEBIT     | received      | INFO      |
+----------------+---------+-----------+---------------+-----------+
1 row in set (0.010 sec)
```

*SELECT with JOINs — 1 row: transaction_id=8, amount=5000.00, DEBIT, category=received, INFO*

## Test 5 — UPDATE Transaction

The test transaction amount is updated from 5000.00 to 9000.00 RWF to verify UPDATE works on the Transactions table.

```
mysql> UPDATE Transactions
    -> SET amount = 9000.00
    -> WHERE momo_tx_id = 99999999999;
Query OK, 1 row affected (0.011 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

*UPDATE Transactions — 1 row matched, 1 changed, 0 warnings*

## Test 6 — INSERT Participant

A Transaction_Participants record is inserted linking the test user to the test transaction with role SENDER.

```
mysql> INSERT INTO Transaction_Participants (transaction_id, user_id, role)
    -> VALUES (LAST_INSERT_ID(), 1, 'SENDER');
Query OK, 1 row affected (0.008 sec)
```

*INSERT INTO Transaction_Participants — Query OK, 1 row affected*

## Test 7 — DELETE Transaction

The test transaction is deleted. Because Transaction_Participants has ON DELETE CASCADE, any participant records linked to this transaction are automatically deleted.

```
mysql> DELETE FROM Transactions
    -> WHERE momo_tx_id = 99999999999;
Query OK, 1 row affected (0.007 sec)
```

*DELETE FROM Transactions — Query OK, 1 row affected (0.007 sec)*

## Test 8 — Confirm CASCADE Delete

A SELECT on Transaction_Participants confirms that the CASCADE worked correctly. The test transaction's participant was automatically removed when the transaction was deleted. Only the original participants for transaction_id = 1 remain.

```
mysql> SELECT * FROM Transaction_Participants
    -> WHERE transaction_id = 1;
+----------------+----------------+---------+----------+---------------------+
| participant_id | transaction_id | user_id | role     | created_at          |
+----------------+----------------+---------+----------+---------------------+
|              2 |              1 |       1 | RECEIVER | 2026-05-18 15:50:38 |
|              1 |              1 |       2 | SENDER   | 2026-05-18 15:50:38 |
+----------------+----------------+---------+----------+---------------------+
2 rows in set (0.011 sec)
```

*SELECT Transaction_Participants WHERE transaction_id = 1 — original data intact, test data cascaded*